Diseño de aplicaciones con UML

¿Qué es UML

UML (Lenguaje unificado de modelado) es un lenguaje que permite representar el diseño y estructura de un Proyecto de Software bajo el paradigma de orientación a objetos.

Define los elementos y reglas para diseñar diferentes tipos de diagramas en función del aspecto del proyecto que se quiere representar o modelar:

- **Diagramas de comportamiento**: Se utilizan para representar qué tiene que ocurrir en el sistema que se modela
    - Diagrama de actividad
    - Diagrama de casos de uso*
    - Diagrama de estados
- **Diagramas de estructura**: Se utilizan para representar los componentes que deben existir en el sistema
    - Diagrama de clases*
    - Diagrama de componentes
    - Diagrama de despliegue
- **Diagramas de interacción**: Se utilizan para representar el flujo de la información del sistema
    - Diagrama de secuencia*
    - Diagrama de colaboración

Herramientas para el diseño de Diagramas UML

- diagrams diagrams.net (draw.io) *[windows|linux|osx]*
- StarUML StarUML *[windows|linux|osx]*
- Modelio Modelio *[windows|linux|osx]*

Diagramas de casos de uso

- Se utilizan para diseñar las diferentes tareas (o funcionalidades) que tiene que despeñar un sistema.
- Se modelan también los actores o entidades externas que interactúan con dicho sistema.
- A cada una de esas tareas o formas en que una actor utiliza el sistema, se le denomina caso de uso.

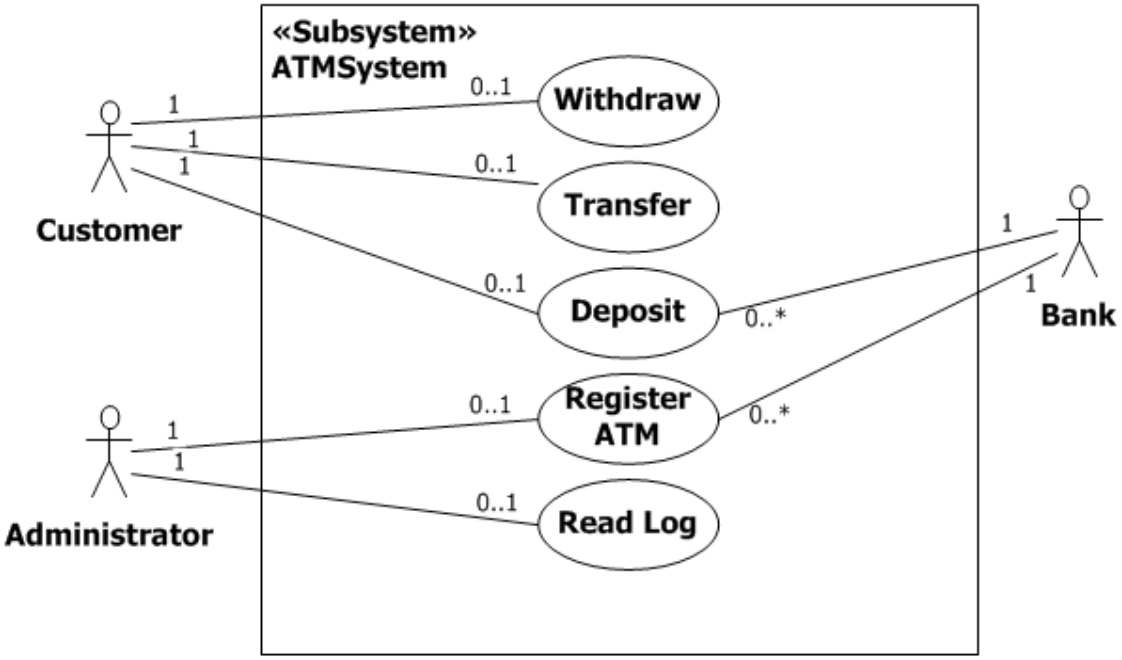


Figure 1: Diagrama de casos de uso

Cómo diseñar diagramas de casos de uso

Elementos de un diagrama de casos de uso:

- **Actor**: Entidad que hace uso del sistema bajo un determinado rol
- **Caso de uso**: Representa una funcionalidad del sistema
- **Límite del sistema**: Define el ámbito del sistema. Define que está y que no está dentro del mismo

Relaciones entre casos de uso:

- **Include**: Permite indicar que un caso de uso utiliza la funcionalidad de otro (siempre lo hace)
- **Extends**: Permite indicar que un caso de uso puede extender la funcionalidad de otro (no necesariamente)
- **Generalization**: Permite indicar que un caso es una variante de otro (“herencia”)

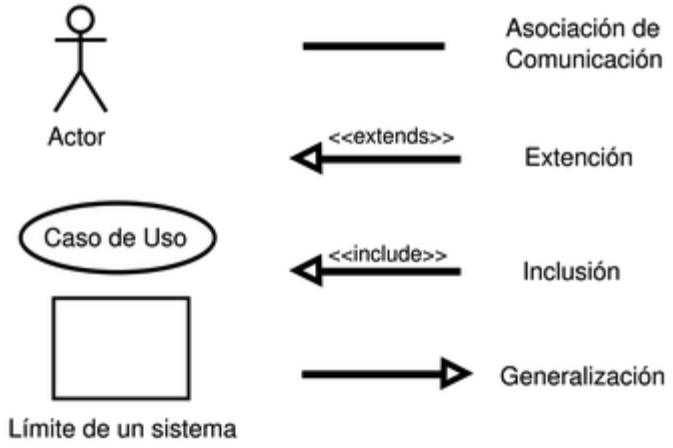


Figure 2: Elementos en un diagrama de casos de uso

Diagramas de clases

- Describe la estructura estática de un sistema
- Se describen las clases que forman el sistema y las relaciones entre éstas
- Además, para cada clase, se definen sus atributos y operaciones (métodos). Normalmente se evita incluir métodos como getters y setters o aquellos que no aportan demasiado significado al modelo

Diagrama de Clases

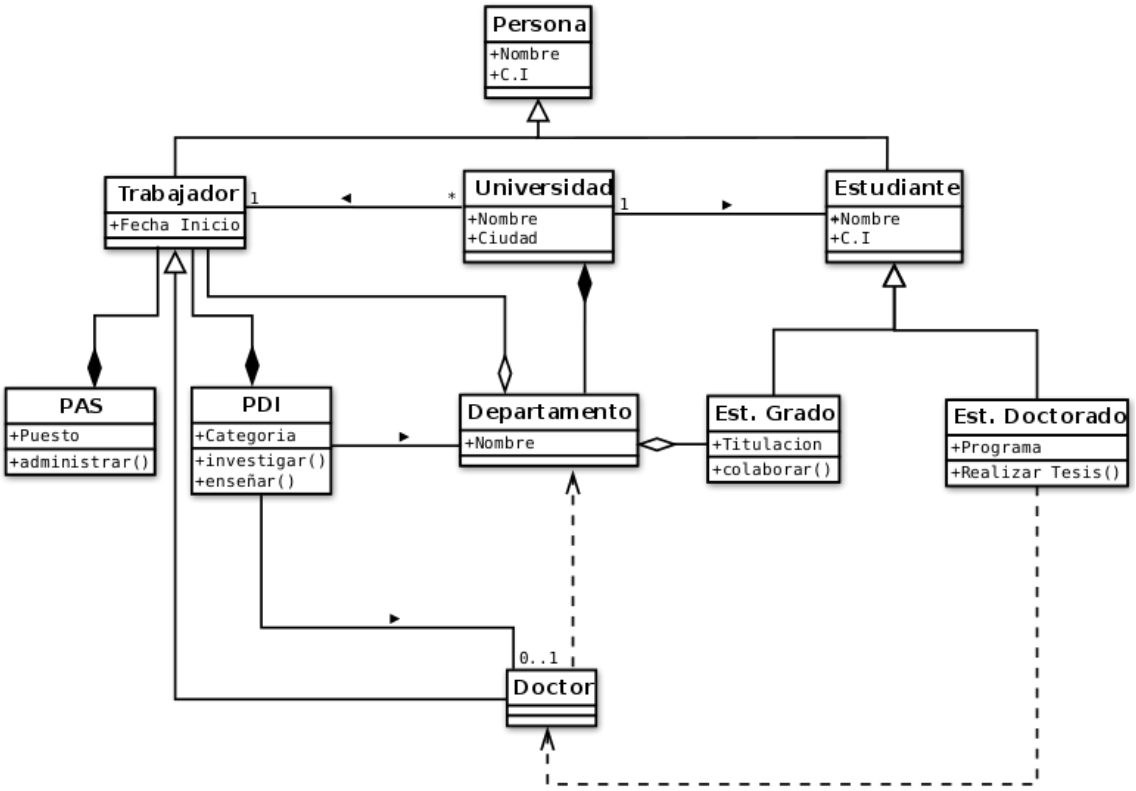


Figure 3: Diagrama de clases

Cómo diseñar diagramas de clases

- Se representan todas las clases (e interfaces) relevantes del sistema
- Se suelen indicar también los atributos y métodos más relevantes de cada clase
- Existe un símbolo para distinguir entre los diferentes tipos de clases: clases, clases abstractas e interfaces
- También deben representarse las relaciones entre las clases, con símbolos que permite especificar el tipo de relación que hay entre ellas (de pertenencia, de uso, . . .)
- Si una clase no es muy relevante, puede no incluirse (clases Utils, constantes, clases poco representativas, . . .)

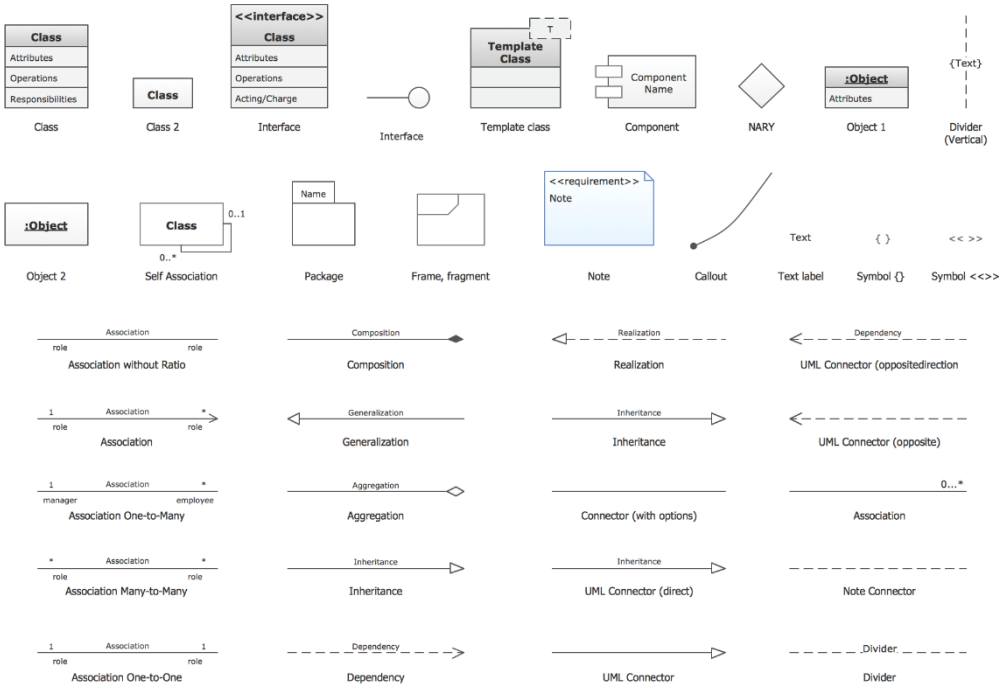
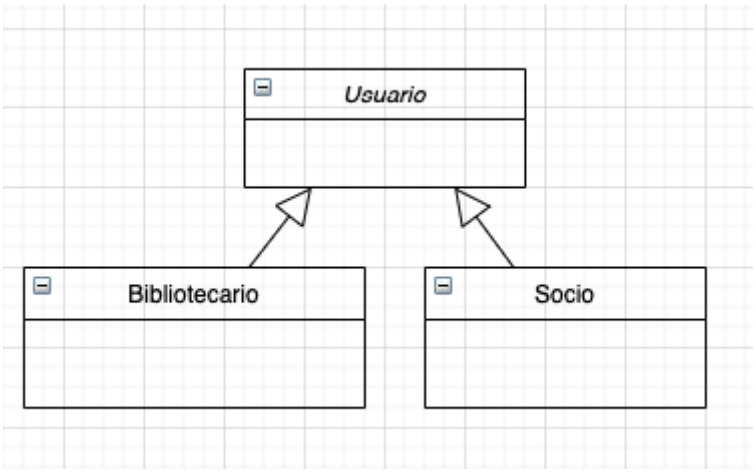


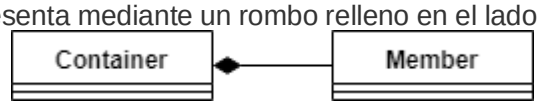
Figure 4: Elementos de un diagrama de clases

Con respecto a las relaciones entre clases:

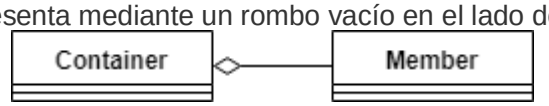
- **Herencia**: Permite representar una relación de herencia entre dos clases
  - Se representa mediante una flecha blanca de la clase que hereda hacia la clase base



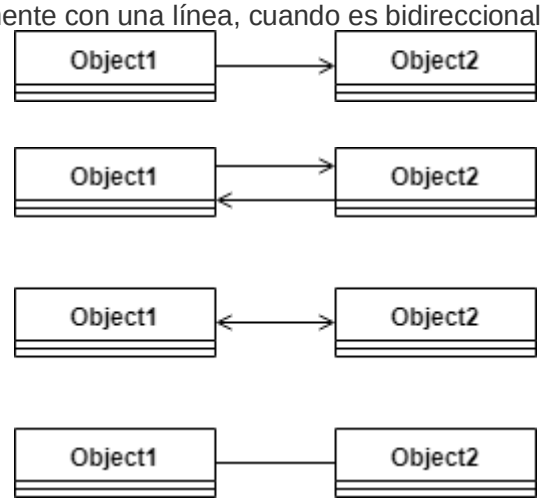
- **Composición**: Permite representar una relación en la que una clase es una parte de otra. En este caso la primera clase no tiene sentido sin que exista la segunda, puesto que forma parte de ella
  - Se representa mediante un rombo relleno en el lado de la clase contenedora



- **Agregación**: Permite representar que una clase está relacionada pero cada una de las clases tiene su propio ciclo de vida. Es una relación más débil que la anterior
  - Se representa mediante un rombo vacío en el lado de la clase Contenedora



- **Asociación**: Representa una relación más débil. Ninguno de los elementos son parte del otro
  - Se representa mediante una flecha hacia la clase con la que se relaciona cuando es unidireccional o bien con dos flechas (una a cada lado) o simplemente con una línea, cuando es bidireccional



Hay un artículo muy interesante sobre las relaciones entre clases en https://www.baeldung.com/java-composition-aggregation-association

---

Diagramas de secuencia

Describe el flujo de información en la interacción entre diferentes objetos de un sistema
Permite representar la secuencia de llamadas a métodos entre objetos que están relacionadas entre sí para definir un proceso de negocio del sistema

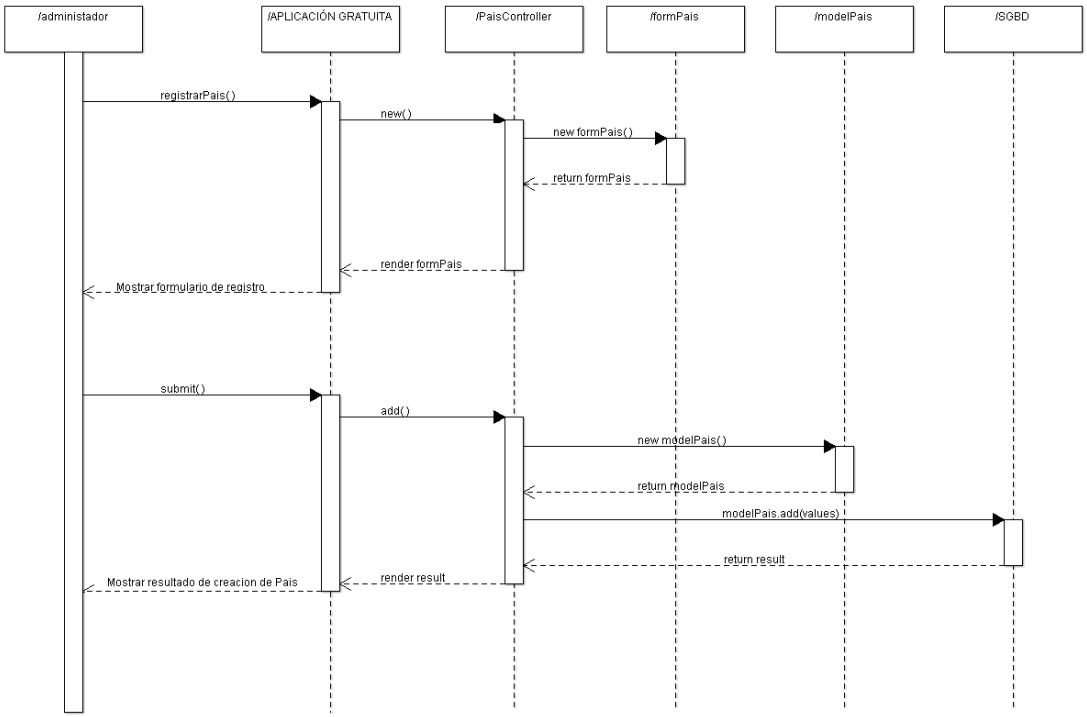


Figure 5: Diagrama de secuencia

Cómo diseñar diagramas de secuencia

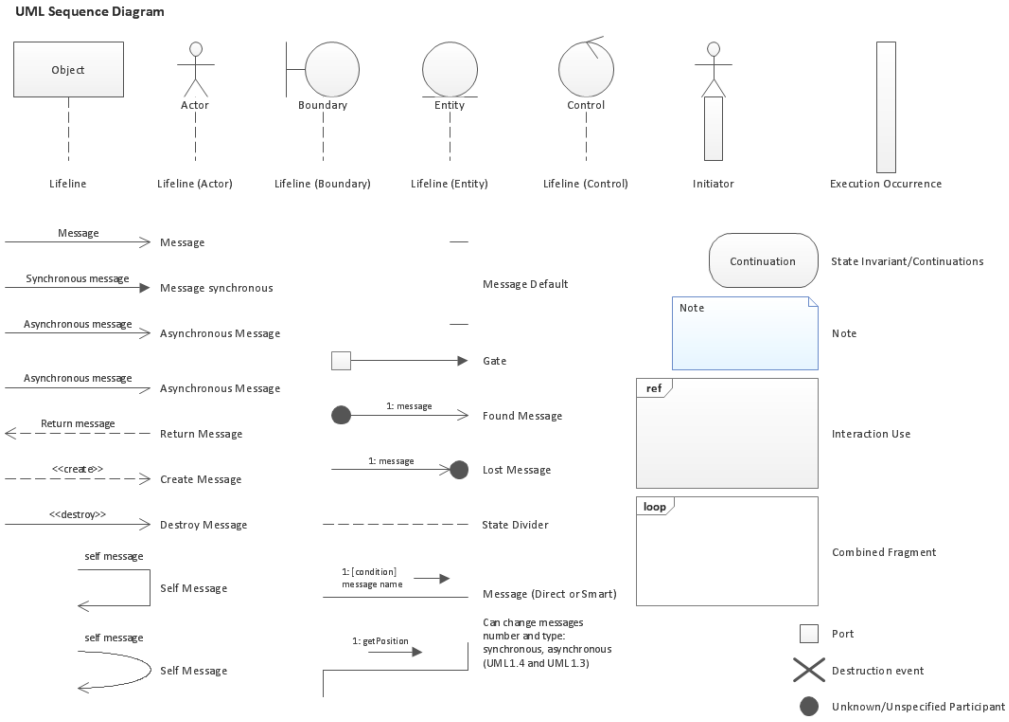


Figure 6: Elementos de un diagrama de secuencia